Міністерство освіти і науки України
Кам’янець-Подільський національний університет імені Івана Огієнка
Фізико-математичний факультет
Кафедра комп’ютерних наук

КУРСОВА РОБОТА
з дисципліни «Об’єктно-
орієнтоване програмування »
на тему:
«Розробка системи управління документами з використанням паттерну Прототип »

студента 2 курсу KNms1-B23 групи
спеціальності 122 Комп’ютерні науки
Горецького Максима Ігоровича
Керівник: к. т. н., доцент кафедри
комп’ютерних наук,
Слободянюк Олександр Васильович

Національна шкала_____
Кількість балів:____ Оцінка ЄКТС: ____

Члени комісії: _____ _____
(підпис) (прізвище та ініціали)
_____ _____
(підпис) (прізвище та ініціали)
_____ _____
(підпис) (прізвище та ініціали)

Кам’янець-Подільський, 2025

ЗМІСТ

ВСТУП 3
РОЗДІЛ 1. ТЕОРЕТИЧНА ЧАСТИНА 5
1.1. Аналіз предметної області 5
1.2. Проєктування системи 6
РОЗДІЛ 2. ПРАКТИЧНА ЧАСТИНА 11
2.1. Опис реалізації 11
2.2. Інтерфейс користувача 16
2.3. Тестування та налагодження 21
ВИСНОВКИ 24
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 26
Додатки 27

ВСТУП

У сучасних умовах цифрової трансформації суспільства все більшу роль відіграють автоматизовані програмні засоби, що забезпечують ефективне управління інформаційними потоками. Одним із ключових завдань в офісному середовищі є створення текстових документів – звітів, листів, презентацій, довідок, інструкцій тощо. Часто ці документи мають повторювану структуру, містять схожі елементи й створюються на регулярній основі. У зв'язку з цим актуальним є питання уніфікації, стандартизації та автоматизації процесу підготовки офісної документації.

Розробка програмних систем, що дозволяють швидко створювати документи різних типів за допомогою шаблонів, сприяє підвищенню продуктивності праці працівників, зменшенню кількості помилок при введенні інформації та забезпеченню єдиного підходу до оформлення документів у межах організації. Особливої цінності набуває можливість збереження таких документів у універсальних форматах (наприклад, XML або JSON), що дозволяє інтегрувати створену систему з іншими інформаційними ресурсами підприємства.

Об'єктом дослідження є процес автоматизованого створення офісних документів у програмному середовищі.

Предметом дослідження виступають методи програмної реалізації генерації документів різних типів з урахуванням їх специфіки та вимог до форматування, з використанням шаблонів і стандартів збереження даних.

Метою курсової роботи є створення програмного засобу для генерації документів типу «звіт», «лист» і «презентація» з урахуванням специфічних атрибутів кожного документа, реалізацією шаблону проєктування «Фабрика» та підтримкою експорту в формати XML і JSON.

Для досягнення поставленої мети в курсовій роботі вирішуються такі завдання:

- дослідити особливості побудови системи створення офісних документів;

- розробити ієрархію класів на основі принципів об’єктно-орієнтованого програмування;
- реалізувати шаблони проєктування для генерації об’єктів документів;
- забезпечити підтримку збереження документів у форматах JSON та XML;
- створити зручний інтерфейс користувача для взаємодії з системою;
- перевірити коректність функціонування програми шляхом тестування.

Курсова робота складається зі вступу, двох основних розділів, висновків, списку використаних джерел та додатків. У **першому розділі** розглянуто теоретичні основи побудови системи: аналіз предметної області, обґрунтування вибору архітектурного шаблону, опис застосованих патернів проєктування та принципів ООП. У **другому розділі** наведено реалізацію програмного продукту: опис інтерфейсу, логіки створення документів, збереження та завантаження, приклади тестування та результати перевірки працездатності системи. У **висновках** підсумовано результати роботи, наведено труднощі, що виникли в процесі реалізації, та можливі напрямки подальшого розвитку.

РОЗДІЛ 1. ТЕОРЕТИЧНА ЧАСТИНА

1.1 Аналіз предметної області

У рамках курсової роботи розробляється система створення офісних документів, яка дозволяє користувачу обирати тип документа, задавати його параметри та зберігати у форматах XML або JSON. Така система може бути застосована в офісах, де щодня потрібно створювати уніфіковані документи з визначеною структурою.

Умовно предметну область можна поділити на основні компоненти, кожен з яких має своє призначення.

1. **Типи документів**

   Система повинна підтримувати кілька основних типів документів:

- Звіт (Report) - містить заголовок, автора, дату створення, список розділів або короткий зміст.
- Лист (Letter) - включає звернення, основний текст, одержувача та підпис.
- Презентація (Presentation) - складається з назви, списку слайдів (їх заголовків або короткого опису).

Кожен тип має специфічні поля, але водночас успадковує спільну базову структуру - наприклад, заголовок, автор і дата.

2. **Створення документів**

   Користувач має можливість обрати тип документа з переліку, після чого заповнити поля, що відповідають обраному типу. Після заповнення даних документ створюється в пам’яті програми і зберігається у вибраному форматі.

3. **Формати збереження**

   Дані документів зберігаються у форматах JSON та XML, що дає змогу:

- Зчитувати їх у зовнішніх системах;
- Імпортувати для подальшого редагування;
- Вести структурований архів створених документів.

4. **Завантаження шаблонів**

Користувач також може завантажити шаблон документа з

конфігураційного файлу, що автоматично заповнить відповідні поля. Це особливо корисно для компаній, де структура документів завжди однакова.

1.2 Проєктування системи

У цьому розділі розглядається архітектура програмної системи створення документів для офісу, її основні компоненти, застосовані шаблони проєктування, структура класів та ключові сценарії використання. Основна мета архітектури - забезпечити модульність, масштабованість і зручність розширення системи, зокрема можливість легко додавати нові типи документів.

1.2.1 Використання принципів об’єктно-орієнтованого програмування

При проєктуванні системи створення офісних документів були застосовані ключові принципи об'єктно-орієнтованого програмування: **інкапсуляція**, **наслідування**, **поліморфізм** та **абстракція**. Вони забезпечили логічну організацію коду, повторне використання компонентів, гнучкість при зміні вимог і простоту масштабування системи.

Інкапсуляція

Вся логіка, що стосується конкретного типу документа, зосереджена в межах відповідного класу. Спільні дані, як-от заголовок, автор і дата створення, інкапсульовано в базовому класі документа. Специфічні властивості (наприклад, тема презентації або терміновість листа) винесено у відповідні похідні класи.

Використання властивостей і приватних полів забезпечує контроль доступу до даних та підтримку узгодженого стану об’єктів. Також у ViewModel доступ до полів реалізується через публічні API, що дозволяє дотримуватися принципу єдиної відповідальності.

Наслідування

Ключовим елементом структури є абстрактний клас документа, який визначає основні спільні характеристики всіх документів. Класи Report, Letter та Presentation наслідують його, розширюючи функціональність відповідно до специфіки кожного типу.

Такий підхід дозволяє уникнути дублювання коду та централізовано керувати загальними аспектами документів. У майбутньому це значно полегшує додавання нових типів документів.

Поліморфізм

Система спроєктована так, що робота з документами здійснюється через посилання на базовий тип. Наприклад, під час серіалізації або відображення деталей у вікні користувач бачить результат методу, спільного для всіх документів, але реалізованого по-різному для кожного типу.

Це забезпечує гнучкість: програма не потребує знати точний тип документа, щоб коректно з ним працювати. Достатньо викликати спільний метод, який реалізує очікувану поведінку згідно з типом.

Абстракція

Абстрактний клас документа та використання фабричних методів є прикладами ефективного застосування абстракції. Клас DocumentFactory приховує логіку створення конкретного типу документа від зовнішнього коду. Це дозволяє користувачу або іншій частині системи працювати з документами через спільний інтерфейс, не вдаючись до деталей реалізації.

Також важливою частиною абстракції є архітектурний шаблон MVVM, у якому кожен компонент виконує свою чітко визначену роль: View - інтерфейс, ViewModel - логіка, Model - дані. Це створює відокремлення обов’язків та забезпечує незалежність модулів.

1.2.2 Опис архітектури (MVVM)

Система побудована за принципами шаблону MVVM (Model-View-ViewModel), який дозволяє розділити логіку інтерфейсу, бізнес-логіку та дані. Це забезпечує кращу підтримку, тестування та повторне використання компонентів.

- **View** - графічний інтерфейс користувача, реалізований у XAML, відображає дані, які надходять із ViewModel, та надсилає команди.

- **ViewModel** - посередник між View та Model. Тут реалізовано логіку керування введенням, валідацію даних, створення та збереження документів.
- **Model** - класи, що описують структуру документів, а також серіалізацію у формати JSON та XML.

Архітектура також передбачає використання фабричних класів для створення об'єктів документів - реалізацію шаблону "Фабрика".

1.2.3 Використання шаблонів проєктування

Factory Pattern

Патерн "Фабрика" дозволяє створювати об'єкти різних типів документів (звіт, лист, презентація) через єдиний інтерфейс. Клас DocumentFactory надає методи:

Таблиця 1.1

Методи фабрики для створення типів документів

Тип документа	Метод фабрики	Опис
Звіт	CreateReport()	Створення об'єкта звіту
Лист	CreateLetter()	Створення об'єкта листа
Презентація	CreatePresentation()	Створення об'єкта презентації

Abstract Factory

З метою гнучкості використовується також шаблон "Абстрактна фабрика". Це дозволяє легко змінювати реалізації без зміни основного коду.

Використання шаблону «Абстрактна фабрика» для створення документів

Таблиця 1.2

Використання шаблону «Абстрактна фабрика» для створення документів

Тип документа	Інтерфейс фабрики	Реалізація
Звіт	IDocumentFactory	ReportFactory
Лист	IDocumentFactory	LetterFactory
Презентація	IDocumentFactory	PresentationFactory

Опис класів

- **Document (abstract)**

  Містить загальні властивості: Title, Author, CreatedDate, Content.
- **Report**

  Додаткові поля: Department, Summary, PageCount.
- **Letter**

  Додаткові поля: Recipient, Subject, IsUrgent.
- **Presentation**

  Додаткові поля: Theme, TargetAudience, SlideCount.
- **DocumentFactory**

  Має методи створення об'єктів різних типів.
- **JSONSerializer/XMLSerializer**

  Забезпечують серіалізацію та десеріалізацію документів.

Таблиця з описом полів

Таблиця 1.3

Таблиця опису класів

Клас	Поле	Тип	Опис
Document	Title	string	Назва документа
Document	Author	string	Автор документа
Document	CreatedDate	DateTime	Дата створення
Report	Department	string	Відділ, який створює звіт
Report	Summary	string	Короткий опис
Report	PageCount	int	Кількість сторінок
Letter	Recipient	string	Отримувач листа
Letter	Subject	string	Тема листа
Letter	IsUrgent	bool	Позначення терміновості
Presentation	Theme	string	Тема презентації
Presentation	TargetAudience	string	Цільова аудиторія
Presentation	SlideCount	int	Кількість слайдів

1.2.4 Валідація введених даних

У ViewModel реалізовано логіку перевірки правильності заповнення полів перед створенням документа.

- **Загальні перевірки**: Title і Author не повинні бути порожніми.
- **Для звіту (Report)**: обов’язковим є поле Department.
- **Для листа (Letter)**: обов’язкові поля Recipient і Subject.
- **Для презентації (Presentation)**: необхідно вказати Theme і TargetAudience.

У разі помилки виводиться повідомлення, а створення документа не виконується. Такий підхід дозволяє мінімізувати помилки користувача та підвищити надійність системи.

1.2.5 Сценарії використання

У цьому розділі ми розглянемо, як користувач взаємодіє з програмою для створення документів. Для цього можна описати кілька основних сценаріїв, що відповідають різним діям користувача.

1. **Вибір типу документа**: Користувач вибирає тип документа через інтерфейс програми, наприклад, через кнопку чи випадаючий список (Звіт, Лист або Презентація).
2. **Заповнення полів документа**: Користувач заповнює необхідні поля документа. Для звіту це можуть бути розділи і зміст, для листа - звернення і підпис, а для презентації - назви слайдів і їх зміст.
3. **Створення документа**: Користувач натискає кнопку "Створити документ", і система, використовуючи відповідну фабрику (DocumentFactory), створює об'єкт потрібного типу.
4. **Збереження документа**: Після того, як користувач заповнив всі поля, програма надає можливість зберегти документ у форматах **XML** або **JSON**. Користувач вибирає формат і місце збереження.

Перед створенням документа система перевіряє, щоб обов’язкові поля не були порожніми. Залежно від обраного типу документа (звіт, лист, презентація), перевіряються й відповідні специфічні поля. У разі помилки, користувачу виводиться відповідне повідомлення без створення документа.

РОЗДІЛ 2. ПРАКТИЧНА ЧАСТИНА

2.1 Опис реалізації

2.1.1 Структура проєкту

Проєкт реалізовано за архітектурою MVVM (Model-View-ViewModel), що дозволяє чітко розділяти логіку додатку, його інтерфейс та дані. Основні компоненти проєкту зображені нижче:

Моделі (Models)

- **Document.cs** - базовий клас для представлення загальної структури документа. Містить основні властивості документа, такі як Title, Author, CreatedDate.
- **Report.cs, Letter.cs, Presentation.cs** - класи, що успадковують від класу Document і представляють специфічні типи документів. Кожен з цих класів містить властивості, які є характерними для відповідного типу документа. Наприклад:
  - **Report** має властивості Department, Summary, PageCount.
  - **Letter** має властивості Recipient, Subject, IsUrgent.
  - **Presentation** має властивості Theme, TargetAudience, SlideCount.
- **DocumentFactory.cs** - клас, який реалізує патерн "Фабрика" для створення об'єктів різних типів документів (Report, Letter, Presentation).

Сервіси (Services)

- **DocumentService.cs** - клас, відповідальний за збереження, завантаження та серіалізацію документів у різні формати. Він реалізує методи для збереження документів у форматах JSON та XML, а також для завантаження документів із цих форматів.

ViewModel

- **MainViewModel.cs** - основний клас ViewModel, який відповідає за обробку логіки інтерфейсу користувача. Він включає властивості для роботи з документами, такі як Title, Author, Recipient, а також команди для створення документів, їх збереження та завантаження. Взаємодіє з сервісами та моделями для керування станом і виведення даних у View.

- **ViewModelBase.cs** - базовий клас для всіх ViewModel, що містить загальні функції для керування властивостями та оповіщенням про їх зміни.

Перегляд (Views)

- **MainWindow.xaml / MainWindow.xaml.cs** - головне вікно програми, що містить інтерфейс для введення даних та вибору типу документа. Вікно взаємодіє з ViewModel для отримання та відображення даних. Код зазвичай містить обробники подій і зв'язок між інтерфейсом і ViewModel.
- **DocumentDetailsWindow.xaml / DocumentDetailsWindow.xaml.cs** - вікно, яке відображає деталі документа, такі як заголовок, автор, специфічні поля для типу документа. Використовується для перегляду збережених документів. Знову ж таки, зв'язок між інтерфейсом і ViewModel забезпечує функціональність вікна.

App.xaml / App.xaml.cs

- **App.xaml** - файл, який визначає налаштування додатку, включаючи стиль, ресурси та стартову точку для виконання додатку.
- **App.xaml.cs** - кодовий файл, який реалізує основну логіку додатку, наприклад, ініціалізацію головного вікна та управління життєвим циклом програми.

2.1.2 Приклад створення документа

Для створення документів різних типів (звіт, лист, презентація) реалізовано механізм через **ViewModel**. Створення документу передбачає використання **DocumentFactory**, яка забезпечує можливість створювати об'єкти різних типів на основі вибору користувача. Розглянемо цей процес на прикладі **MainViewModel**:

1. **Вибір типу документа**:

Користувач обирає тип документа через властивість SelectedDocumentType, яка налаштовує видимість полів для кожного з типів (Report, Letter, Presentation).

Реалізація методу SelectedDocumentType() представлена у Додатку А.

Це забезпечує динамічну зміну доступних полів на основі обраного типу документа.

2. **Створення документа**:

При натисканні на кнопку для створення документа викликається метод CreateDocument, в якому здійснюється:

- Валідація загальних полів документа (Title, Author).
- Створення документа через **DocumentFactory** на основі вибраного типу.
- Заповнення специфічних полів в залежності від типу документа (наприклад, для звіту додаються відділ, підсумок та кількість сторінок, для листа - отримувач, тема, терміновість).

Реалізація методу CreateDocument() представлена у Додатку Б.

3. **Звіт, лист і презентація**:

Для кожного з типів документів передбачена власна специфікація полів:

- **Report**: Включає відділ, підсумок та кількість сторінок.
- **Letter**: Включає отримувача, тему та маркер терміновості.
- **Presentation**: Включає тему, цільову аудиторію та кількість слайдів.

Кожен документ містить метод GetDetails, який виводить детальну інформацію про документ у текстовому форматі:

Реалізація методу SelectedDocumentType() представлена у Додатку В.

Виведення інформації:

Після створення документа користувач може побачити результат у вікні програми або через команду "Показати деталі". Для цього використовується окреме вікно, що відображає всі дані про документ:

Реалізація методу ShowDocumentDetails() представлена у Додатку Г.

Таким чином, створення документа у вашому додатку є інтерактивним процесом, що включає вибір типу, введення загальних і специфічних даних та перевірку правильності введення перед формуванням документа.

2.1.3 Збереження в JSON/XML

У рамках реалізації функціональності збереження документів передбачено підтримку двох форматів: **JSON** та **XML**. Для серіалізації об'єкта документа у формат JSON використовується бібліотека **Newtonsoft.Json**. З метою

забезпечення збереження повної інформації про тип документа, налаштовано параметр TypeNameHandling.All, який дозволяє коректно серіалізувати об'єкти похідних класів (Report, Letter, Presentation). Вивід JSON структурований за допомогою параметра Formatting.Indented.

Фрагмент коду, що реалізує збереження документа у форматі JSON, наведено у Додатку Д.

Для завантаження документа з JSON-файлу застосовується зворотна операція десеріалізації з аналогічними налаштуваннями:

Фрагмент коду, що відповідає за завантаження документа з JSON-файлу, подано у Додатку Д.

Збереження у форматі XML

Серіалізація в XML реалізована за допомогою класу XmlSerializer. Оскільки XML не підтримує автоматичне збереження інформації про тип об'єкта, під час завантаження необхідно явно вказувати тип документа (наприклад, Report, Letter, Presentation).

Фрагмент коду для збереження документа у форматі XML розміщено у Додатку Е.

Під час десеріалізації XML-документа тип визначається на основі вхідного параметра documentType, після чого застосовується відповідний десеріалізатор:

Фрагмент коду, що реалізує завантаження документа з XML-файлу, наведено у Додатку Е.

2.1.4 Завантаження шаблонів

У програмі реалізовано функціональність завантаження шаблонів документів. Шаблони зберігаються у форматі JSON або XML і містять попередньо заповнені значення для відповідного типу документа. Це дозволяє користувачу швидко створювати типові документи, завантажуючи шаблон із файлу.

Завантаження шаблону виконується за допомогою методів LoadFromJson або LoadFromXml класу DocumentService, які повертають об'єкт типу Document із заповненими полями.

Фрагмент коду, що реалізує завантаження документа з JSON-файлу, наведено у Додатку Є.

Після вибору шаблону (наприклад, template_letter.json) за допомогою діалогу відкриття файлу, вміст шаблону завантажується в об'єкт документа, і відповідні значення відображаються у спливаючому вікні.

2.1.5 Очищення полів, вивід інформації

У додатку реалізовано можливість очищення полів форми введення, що дозволяє швидко скинути всі дані користувача. Для цього у ViewModel реалізовано метод ClearFields, який прив’язаний до відповідної кнопки у графічному інтерфейсі. Метод встановлює всі поля, що відображаються у формі, до значень за замовчуванням (порожній рядок або 0/false для числових і логічних полів). Також встановлюється службове повідомлення про успішне очищення:

Фрагмент коду, що реалізує очищення полів документа та встановлення повідомлення про стан, наведено у Додатку Ж.

Окрім цього, у додатку передбачено можливість виводу інформації про створений або завантажений документ. Деталі документа відображаються у відповідному вікні або панелі - залежно від реалізації View. Цей функціонал забезпечується двостороннім зв’язком між ViewModel і View згідно з шаблоном MVVM. Дані з властивостей моделі автоматично оновлюються у графічному інтерфейсі за допомогою прив’язок (Binding).

2.1.6 Приклад одного збереженого файлу (JSON/XML)

Нижче наведено приклад збереженого документа у форматі **JSON**. Для серіалізації використовувалася бібліотека **Newtonsoft.Json** з увімкненим TypeNameHandling.All, що дозволяє зберігати інформацію про тип об’єкта:

JSON

```
{
  "$type": "DocumentFactory.Models.Report, DocumentFactoryApp",
  "Department": "фізмат",
  "Summary": "перший проект",
  "PageCount": 2,
```

"Title": "Проект",
"Author": "Горецький",
"CreatedDate": "2025-05-13T20:52:15.3568842+03:00"
}

Також наведемо приклад того ж самого документа у форматі **XML**. Для серіалізації використовувався клас XmlSerializer, який зберігає структуру об’єкта як XML-дерево.

XML

<Report xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema">
<Title>Проект</Title>
<Author>Горецький</Author>
<CreatedDate>2025-05-13T20:52:15.3568842+03:00</CreatedDate>
<Department>фізмат</Department>
<Summary>перший проект</Summary>
<PageCount>2</PageCount>
</Report>

Формати зберігання дозволяють зручно передавати, зберігати та відновлювати дані про документи, що створюються у застосунку.

2.2 Інтерфейс користувача

Інтерфейс користувача розроблений із застосуванням принципів мінімалістичного дизайну. Усі елементи мають чітке розташування і відповідні розміри для забезпечення комфортної роботи. Вибір шрифтів, кольорів та стилів елементів інтерфейсу орієнтовано на зручність користувача, з акцентом на легкість у використанні та зрозумілість.

Опис елементів інтерфейсу

Вікно починається з заголовку "Створення документів", який є основною орієнтацією для користувача. Під заголовком відображається коротка інструкція: "Виберіть тип документа та заповніть необхідні поля". Це допомагає

користувачеві швидко зрозуміти, що необхідно зробити для створення нового документа. (див рис. 2.1).

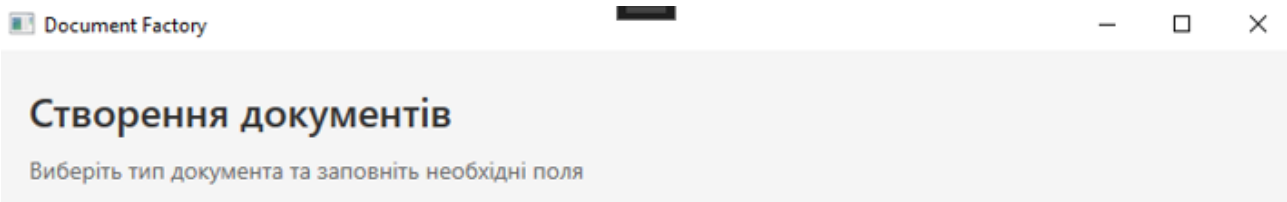

Рис 2.1. Створення документів

Для початку роботи користувач має вибрати тип документа. Це здійснюється за допомогою **ComboBox**, який містить список доступних типів документів: "Звіт", "Лист" та "Презентація". Після вибору відповідного типу відображаються лише ті поля, які є релевантними для вибраного документа. (див. рис. 2.2).

Рис 2.2. Вибір типу документа документів

Усі типи документів мають спільні поля, такі як:

- **Назва документа**
- **Автор документа**

Ці поля представлені у вигляді текстових полів **TextBox**, де користувач може ввести потрібну інформацію. Ці поля є обов'язковими для заповнення, і вони відображаються незалежно від типу документа. (див. рис. 2.3).

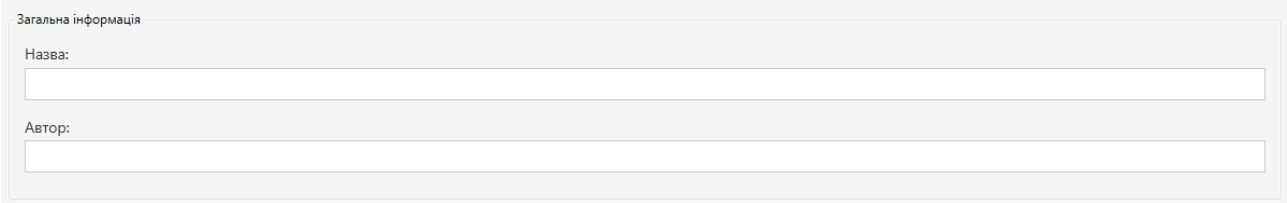

Рис 2.3. Загальні поля для всіх типів документів

Якщо користувач вибирає тип документа "Звіт", відображаються специфічні поля:

- **Відділ:** Текстове поле для введення назви відділу, до якого належить документ.

- **Резюме:** Текстове поле для введення короткої характеристики документа.
- **Кількість сторінок:** Числове поле для введення кількості сторінок у документі.

Ці поля представлені в груповому блоці **GroupBox** з заголовком "Дані звіту". Цей блок стає видимим лише після вибору типу "Звіт". (див. рис. 2.4).

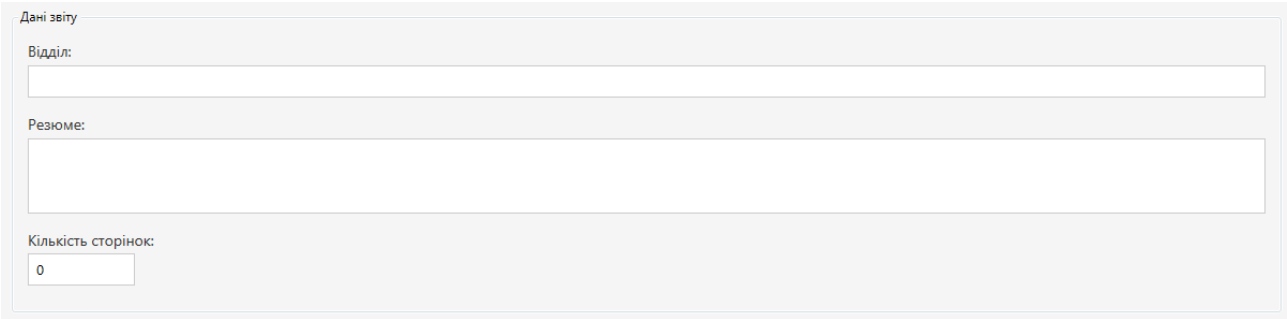

Рис 2.4. Поля для Звіту (Report)

Для створення документа типу "Лист" передбачено спеціальні поля:

- **Отримувач:** Текстове поле для введення імені або адресата листа.
- **Тема:** Текстове поле для введення теми листа.
- **Чекбокс "Терміновий лист":** Це поле дозволяє позначити лист як терміновий.

Блок для листа активується при виборі відповідного типу документа, і всі ці поля стають доступними для редагування. (див. рис. 2.5).

Рис 2.5. Поля для Листа (Letter)

Для типу документа "Презентація" є такі поля:

- **Тема:** Текстове поле для введення теми презентації.
- **Цільова аудиторія:** Текстове поле для введення цільової аудиторії презентації.

- **Кількість слайдів:** Числове поле для введення кількості слайдів у презентації.

Цей блок також відображається лише після вибору типу "Презентація". (див. рис. 2.6).

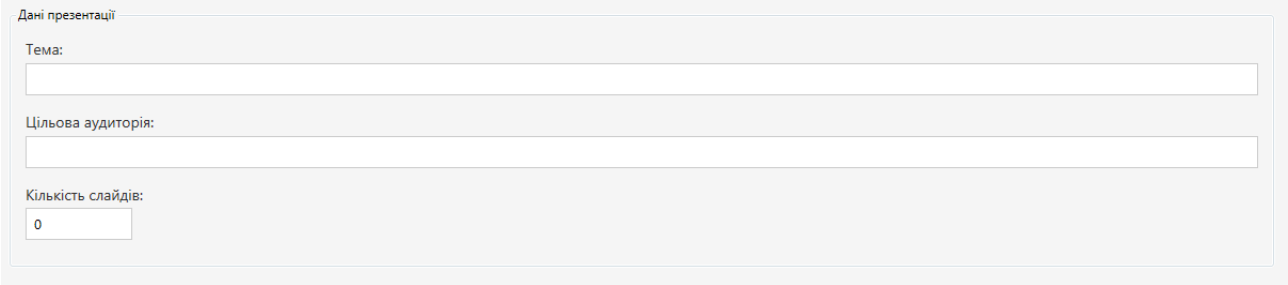

Рис 2.6. Поля для Презентації (Presentation)

Внизу вікна знаходиться панель з кнопками для виконання основних операцій:

1. **Створити:** Кнопка для створення нового документа.
2. **Деталі:** Кнопка для перегляду деталей створеного документа.
3. **Очистити:** Кнопка для очищення всіх полів.
4. **Зберегти як JSON:** Кнопка для збереження документа у форматі JSON.
5. **Зберегти як XML:** Кнопка для збереження документа у форматі XML.
6. **Завантажити з JSON:** Кнопка для завантаження документа з файлу JSON.
7. **Завантажити з XML:** Кнопка для завантаження документа з файлу XML.

Кожна кнопка має стиль, який змінюється при наведенні миші (зміна кольору). Кнопки дають змогу користувачу працювати з документами швидко та ефективно. (див. рис. 2.7).

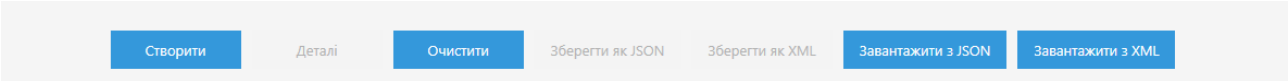

Рис 2.7. Кнопки дій

Загальний вигляд інтерфейсу має наступний вигляд: (див. рис. 2.5).

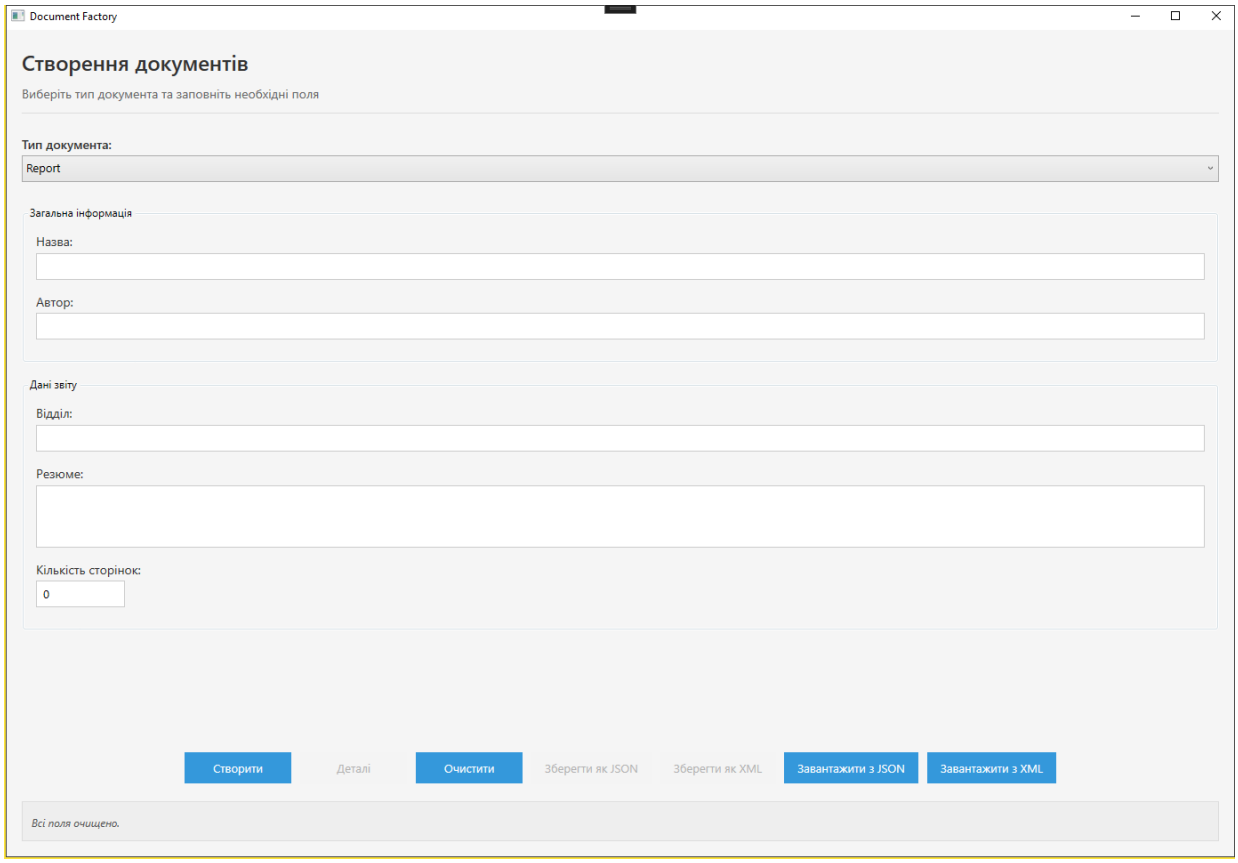

Рис 2.8. Інтерфейс програми

Окреме вікно для деталей документа

Це додаткове вікно, яке відкривається для перегляду повних деталей обраного документа. Вікно містить прокручуване текстове поле, в якому відображаються всі введені дані. Також є кнопка "Закрити", яка дозволяє закрити це вікно після перегляду. (див. рис. 2.8).

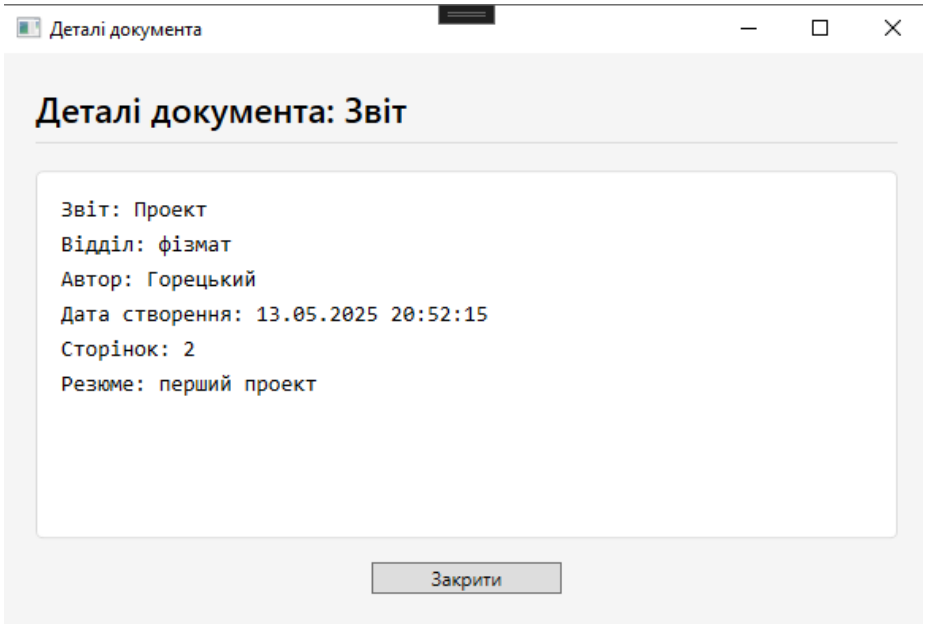

Рис 2.8. Вікно "Деталі документа"

2.3 Тестування та налагодження

2.3.1 Тестування інтерфейсу користувача

Тестування інтерфейсу користувача є важливою частиною розробки для забезпечення коректної роботи всіх елементів і взаємодії користувача з програмою. Це включає перевірку відображення всіх компонентів, правильності реакції на дії користувача та відсутність багів. Для цього були виконані наступні етапи тестування:

1. **Перевірка видимості та доступності елементів інтерфейсу**
   - Перевірено, що всі елементи, такі як текстові поля, кнопки, комбо-бокси, правильно відображаються і знаходяться на своїх місцях.
   - Всі елементи, що мають бути видимими (наприклад, після вибору типу документа), з’являються на екрані згідно з логікою програми.
2. **Перевірка коректної роботи кнопок**
   - Кнопки "Створити", "Очистити", "Зберегти" були протестовані на правильність виконання своїх функцій.
   - Перевірено, що після натискання на кнопку "Очистити", всі введені дані стираються.
   - Тестування кнопок для збереження документів в різних форматах (JSON, XML) підтвердило, що файли коректно зберігаються на диск.
3. **Перевірка взаємодії з вікном "Деталі документа"**
   - Перевірено, що при натисканні на кнопку "Деталі", вікно відкривається і коректно відображає всі дані, що були введені в полях.
   - Вікно закривається після натискання на кнопку "Закрити", не зберігаючи зміни, якщо це не передбачено логікою програми.

2.3.2 Тестування функцій збереження та завантаження

Для перевірки коректності збереження та завантаження документів у форматах JSON та XML були проведені такі етапи:

1. **Збереження документів в JSON та XML форматах**
    - Тестування включало створення документа з різними полями, а потім збереження його у форматі JSON та XML
    - Після збереження, файл перевіряється на наявність всіх полів та коректність їх структури.
    - Проведено тестування з різними типами документів (Звіт, Лист, Презентація) для перевірки, чи правильно зберігаються їх специфічні поля.
2. **Завантаження документів з файлів JSON та XML**
    - Для перевірки функції завантаження було використано різні файли JSON та XML, щоб переконатися в коректному відновленні даних у програмі.
    - Тестування включало перевірку, чи правильно завантажуються дані про тип документа, його поля та виведення на екран. (див. рис. 2.9).

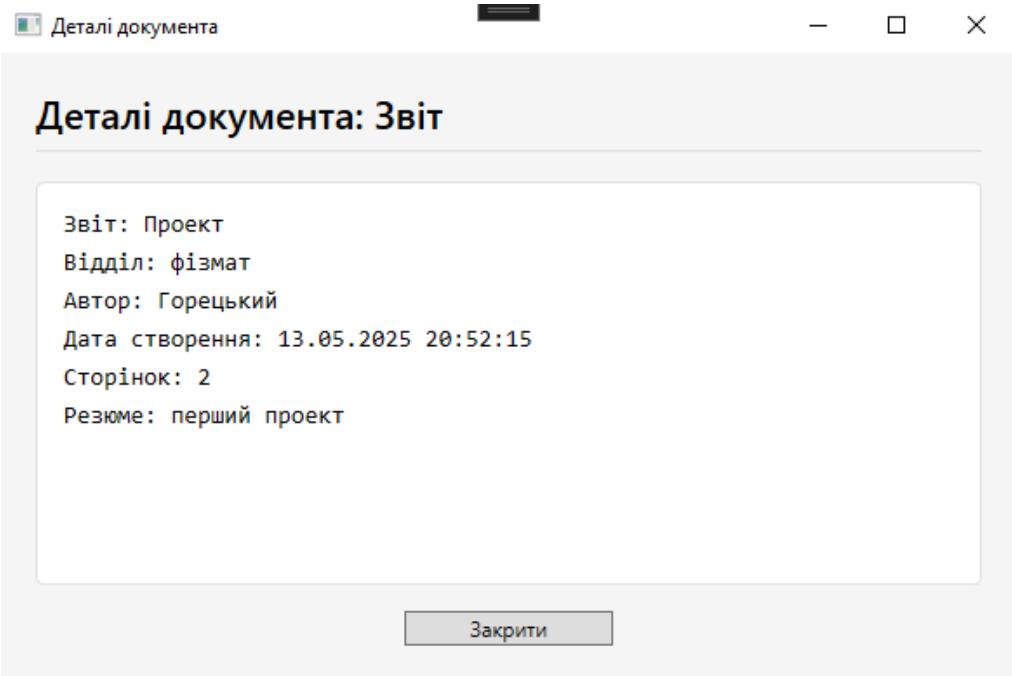

Рис 2.9. Перевіриний вивід інформації

2.3.3 Перевірка на обробку помилок

Один із важливих аспектів тестування - це перевірка обробки помилок при некоректному введенні даних або неправильному форматі файлів. Для цього були перевірені такі сценарії:

Некоректне введення даних

- Якщо користувач не заповнив обов'язкові поля (наприклад, "Назва документа" чи "Автор"), програма повинна відобразити відповідне повідомлення про помилку.
- Якщо користувач намагається зберегти файл у неправильному форматі (наприклад, введення некоректного значення у числове поле), програма повинна вивести попередження.

Помилки при завантаженні файлів

- Перевірено, що програма правильно реагує на намагання завантажити файл з помилковим або пошкодженим форматом.
- Під час завантаження програма перевіряє, чи дійсно вказаний файл містить потрібну структуру даних.

Помилка при завантаженні: Could not create an instance of type DocumentFactory.Models.Document. Type is an interface or abstract class and cannot be instantiated. Path 'runtimeTarget', line 2, position 18.

Рис 2.10. Строка виводу помилок

2.3.4 Тестування продуктивності

З метою перевірки продуктивності програми було проведено наступне тестування:

1. **Час створення документу**
    - Тестування показало, що програма здатна швидко створювати документ навіть при великих обсягах введених даних.
2. **Збереження та завантаження файлів**
    - Перевірено, що збереження документів у форматах JSON та XML виконується без затримок. Також тестувалося завантаження великих файлів, щоб переконатися, що програма справляється з такими завданнями без значних затримок.

2.3.5 Підсумки тестування

У результаті тестування було встановлено, що всі основні функції програми працюють коректно. Жодних серйозних багів або збоїв не було виявлено. Програма успішно справляється з поставленими задачами.

ВИСНОВКИ

У рамках курсової роботи була розроблена система для автоматизації створення офісних документів, що включає такі функції:

- Створення різних типів документів (Звіт, Лист, Презентація) з відповідними полями.
- Валідація введених даних.
- Можливість збереження документів у форматах JSON та XML.
- Завантаження документів з файлів JSON та XML.
- Інтерфейс для введення та перегляду даних про документи.
- Окреме вікно для відображення деталей документа.

Для реалізації цього проєкту було використано наступні технології та інструменти:

- **C#**: основна мова програмування для розробки додатку.
- **WPF (Windows Presentation Foundation)**: для створення графічного інтерфейсу користувача.
- **MVVM (Model-View-ViewModel)**: архітектурний патерн для організації коду та розділення логіки від зовнішнього вигляду.
- **JSON/XML**: для серіалізації та збереження даних документів.
- **XAML**: для опису інтерфейсу користувача.

У процесі розробки були виявлені та подолані кілька труднощів:

1. **Управління складними даними**:
   Розробка та інтеграція різних типів документів з окремими полями для кожного з них вимагала ретельного планування структури даних та їх валідації.
2. **Збереження та завантаження файлів**:
   Проблеми виникли при роботі з різними форматами файлів (JSON і XML), оскільки необхідно було забезпечити коректну серіалізацію та десеріалізацію складних об'єктів.
3. **Інтерфейс користувача**:
   Проектування інтерфейсу так, щоб він був зрозумілим і зручним для

користувача, потребувало кілька ітерацій і тестування для забезпечення найкращого користувацького досвіду.

4. **Обробка помилок**:

   Важливим етапом стало налаштування обробки помилок, щоб у разі некоректного введення даних або завантаження файлів система коректно реагувала та інформувала користувача.

Можливості для подальшого розвитку системи включають:

- **Генерація PDF або .docx**: Додавання можливості збереження документів у форматах PDF або DOCX для підвищення зручності використання.
- **Інтеграція з базами даних**: Розширення для зберігання документів в базі даних, що дозволить зберігати та керувати великими об'ємами даних.
- **Мультимовний інтерфейс**: Додавання підтримки різних мов для розширення можливостей застосунку в міжнародному середовищі.
- **Покращена валідація**: Додавання більш складної валідації даних (наприклад, перевірка дати, формату електронної пошти).
- **Додавання шаблонів**: Можливість використовувати попередньо створені шаблони для різних типів документів, що зменшить час на їх створення.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Громов В. С. Програмування мовою C# у середовищі .NET: навч. посіб. - Харків: ХНУРЕ, 2021. - 312 с.
2. Freeman A., Wagner J. Pro WPF in C# 2022: Windows Presentation Foundation in .NET 6. - Apress, 2022. - 1080 p.
3. Ермолаєв С. А. Архітектурні шаблони проєктування програмного забезпечення: навч. посіб. - Київ: КНУ, 2020. - 180 с.
4. Гамма Е., Хелм Р., Джонсон Р., Вліссідес Д. Шаблони проєктування. Елементи повторно використовуваних об'єктно-орієнтованих програм. - Київ: Діалектика, 2002. - 395 с.
5. Microsoft Docs. Windows Presentation Foundation (WPF) [Електронний ресурс]. - Режим доступу: https://learn.microsoft.com/en-us/dotnet/desktop/wpf/
6. Microsoft Docs. Абстрактні класи і методи (C#) [Електронний ресурс]. - Режим доступу: https://learn.microsoft.com/en-us/dotnet/csharp/programming-guide/classes-and-structs/abstract-and-sealed-classes
7. Newtonsoft.Json Documentation [Електронний ресурс]. - Режим доступу: https://www.newtonsoft.com/json/help/html/Introduction.htm
8. MSDN. XML serialization [Електронний ресурс]. - Режим доступу: https://learn.microsoft.com/en-us/dotnet/standard/serialization/xml-serialization
9. Пономаренко В. С. Основи програмної інженерії. - Харків: ХНУРЕ, 2020. - 210 с.
10. ISO/IEC 29500-1:2016. Information technology - Document description and processing languages - Office Open XML File Formats - Part 1: Fundamentals and Markup Language Reference.

Додатки

Додаток А

```
public string SelectedDocumentType
        {
         get => _selectedDocumentType;
            set
         {
               if (SetProperty(ref _selectedDocumentType, value))
                {
                 IsReportSelected = value == "Report";
                 IsLetterSelected = value == "Letter";
                 IsPresentationSelected = value == "Presentation";

                 ClearFields();
             }
         }
     }
```

Додаток Б

```
private void CreateDocument()
        {
            try
            {
                if (string.IsNullOrWhiteSpace(Title))
                {
                    StatusMessage = "Помилка: Назва документа не може бути
порожньою.";
                    return;
                }

                if (string.IsNullOrWhiteSpace(Author))
                {
                    StatusMessage = "Помилка: Автор документа не може бути
порожнім.";
                    return;
                }
```

```
                    CurrentDocument =
_documentService.CreateDocument(SelectedDocumentType);

                    CurrentDocument.Title = Title;
                    CurrentDocument.Author = Author;
                    CurrentDocument.CreatedDate = DateTime.Now;


                    switch (SelectedDocumentType)
                    {
                        case "Report":
                            report.Department = Department;
                            report.Summary = Summary;
                            report.PageCount = PageCount;
                            break;

                        case "Letter":
                            letter.Recipient = Recipient;
                            letter.Subject = Subject;
                            letter.IsUrgent = IsUrgent;
                            break;
                        case "Presentation":
                            presentation.Theme = Theme;
                            presentation.TargetAudience = TargetAudience;
                            presentation.SlideCount = SlideCount;
                            break;
                       }
                    StatusMessage = $"Документ \"{CurrentDocument.Title}\" успішно
створено.";
                }
```

Додаток В

```
public override string GetDetails()
        {
            return $"Звіт: {Title}\nВідділ: {Department}\nАвтор: {Author}\nДата
створення: {CreatedDate}\nСторінок: {PageCount}\nРезюме: {Summary}";
        }
```

Додаток Г

```
private void ShowDocumentDetails()
        {
            if (CurrentDocument == null) return;

            var detailsWindow = new DocumentDetailsWindow(CurrentDocument);
            detailsWindow.Owner = Application.Current.MainWindow;
            detailsWindow.ShowDialog();
        }
```

Додаток Д

```
public void SaveAsJson(Document document, string filePath)
        {
            var settings = new JsonSerializerSettings
            {
                TypeNameHandling = TypeNameHandling.All,
                Formatting = Formatting.Indented
            };

            string json = JsonConvert.SerializeObject(document, settings);
            File.WriteAllText(filePath, json);
        }

string json = File.ReadAllText(filePath);
            var settings = new JsonSerializerSettings
            {
                TypeNameHandling = TypeNameHandling.All
            };

            return JsonConvert.DeserializeObject<Document>(json, settings);
```

Додаток Е

```
var serializer = new XmlSerializer(document.GetType());
            using (var writer = new StreamWriter(filePath))
            {
                serializer.Serialize(writer, document);
```

```
            }

var serializer = new XmlSerializer(type);
            using (var reader = new StreamReader(filePath))
            {
                return (Document)serializer.Deserialize(reader);
```

Додаток Є

```
 public Document LoadFromJson(string filePath)
        {
            if (!File.Exists(filePath))
                throw new FileNotFoundException($"Файл не знайдено: {filePath}");

            string json = File.ReadAllText(filePath);
            var settings = new JsonSerializerSettings
            {
                TypeNameHandling = TypeNameHandling.All
            };
            return JsonConvert.DeserializeObject<Document>(json, settings);
        }
```

Додаток Ж

```
private void ClearFields()
        {
            Title = string.Empty;
            Author = string.Empty;
            Department = string.Empty;
            Summary = string.Empty;
            PageCount = 0;
            Recipient = string.Empty;
            Subject = string.Empty;
            IsUrgent = false;
            Theme = string.Empty;
            TargetAudience = string.Empty;
            SlideCount = 0;
            StatusMessage = "Всі поля очищено.";
            CurrentDocument = null;
        }
```